VERITY — FAKE NEWS DETECTION SYSTEM (SRS)

1. Introduction and Project Objectives

1.1 Purpose

This Software Requirements Specification (SRS) document provides a formal and detailed description of the design, functional behavioral model, architecture, and operational constraints of 'Verity' — an automated, machine-learning-driven Fake News Detection System. The intent of this document is to serve as the definitive reference during system review, viva evaluation, software integration, and maintenance phases.

1.2 Objectives

• Real-Time Misinformation Identification: Provide instantaneous textual credibility analysis to determine whether input news content is authentic (Real) or fabricated (Fake).
• Modern Modular Frontend: Implement a highly responsive Single Page Application (SPA) using React and Vite, featuring user input validation and stateful response rendering.
• High-Performance API Microservice: Architect an asynchronous RESTful backend service using Python and FastAPI to expose low-latency prediction endpoints.
• Standardized Machine Learning Pipeline: Streamline raw text preprocessing (tokenization, stop-word filtering, noise stripping), numerical vectorization (TF-IDF), and classifier inference using saved model artifacts.
• Academic Scalability & Clean Code Standards: Maintain strict modularity across frontend UI components, API routing controllers, and machine learning inference scripts.

2. Problem Statement & Motivation

In modern digital ecosystems, the rapid proliferation of unverified news articles, social media posts, and sensationalist headlines leads to mass public misinformation, economic manipulation, and societal confusion.

Conventional verification relies heavily on manual fact-checking by human journalists and media analysts. However, manual fact-checking faces severe limitations:
1. Velocity & Scale: Millions of articles are published daily; manual review cannot scale to inspect digital news velocity.
2. Latency: Manual verification requires hours or days, by which time misinformation has already spread.
3. Subjectivity: Human evaluation is vulnerable to cognitive and ideological biases.

Motivation:

Verity addresses these critical vulnerabilities by introducing an algorithmic, sub-second NLP verification

engine that processes text objectively, extracts semantic patterns, and evaluates credibility metrics using pre-trained machine learning models.

3. Scope of the Project

3.1 In-Scope Features & Deliverables

• Text Analysis Web Interface: Single-page responsive React dashboard allowing users to input headlines or full article text.
• Fast API REST Controller: Asynchronous POST endpoint (`/predict`) receiving JSON text payloads.
• Automated Natural Language Preprocessing: Standardization of raw user text via lowercasing, regex cleaning, stop-word elimination, and TF-IDF feature transformation.
• Model Inference Engine: Dynamic loading of serialized machine learning models (`.pkl` / `.joblib`) to execute predictions and return output metrics.
• Real-Time Output Feedback: Displaying prediction tags ('Real News' vs. 'Fake News') along with confidence/probability scores on the UI.

3.2 Out-of-Scope (Future Architectural Enhancements)

• Persistent User Accounts & Query History: Storage of historical queries in a database (e.g., PostgreSQL/MongoDB) is omitted in this stateless iteration.
• Automated Web / Social Media Scraping: Direct fetching of content via external news URLs or social media APIs.
• Multi-modal Deepfake Verification: Analysis of audio, image manipulation, or synthetic deepfake video media.

4. Existing System vs. Proposed System

Existing System (Manual Fact-Checking):

• Process: Human editors search sources, verify claims, and draft counter-reports.
• Throughput: Low (a few articles per analyst per day).
• Latency: High (ranging from several hours to multiple days).
• Consistency: Variable, depending on individual editor expertise and potential bias.

Proposed System (Verity - Automated ML Pipeline):

• Process: Automated text cleaning, feature transformation, and statistical ML model scoring.
• Throughput: High (capable of processing hundreds of API requests per minute).
• Latency: Sub-second (typically < 300 ms end-to-end response time).
• Consistency: 100% deterministic and reproducible algorithmic predictions based on learned corpus features.

5. Detailed Functional Requirements

FR-1 Text Input & Validation:
- The web interface must provide a dedicated multi-line text input field.
- The system shall validate that the input text is non-empty before initiating network calls.

FR-2 REST API Payload Delivery:
- Upon clicking 'Classify', the React application must dispatch an HTTP POST request to `http://<host>:<port>/predict` containing JSON: `{"text": "user input text"}`.

FR-3 Text Preprocessing Module:
- Lowercasing text strings.
- Stripping special characters, punctuation, and HTML tags using regular expressions.
- Removing standard English stop-words via NLTK corpus.
- Transforming cleaned text into numerical vector matrices using pre-fitted TF-IDF vectorizers.

FR-4 Classifier Inference:
- The backend shall pass transformed vectors into the loaded Machine Learning model.
- The model must compute class probability scores and output a discrete prediction ('REAL' or 'FAKE').

FR-5 Results Display & UI State Management:
- The frontend must parse the JSON response: `{"prediction": "REAL", "confidence": 0.94}`.
- The UI must render clear visual indicators (green highlight for Authentic, red highlight for Misleading/Fake).

6. Non-Functional Requirements

NFR-1 Latency & Performance:
- End-to-end API response time (including network transmission, preprocessing, and model inference) must be under 500 ms for inputs up to 2,000 words.

NFR-2 Availability & Reliability:
- The backend service must gracefully catch processing exceptions (e.g., blank strings, invalid JSON) and return standard HTTP 400/422 status codes without crashing Uvicorn.

NFR-3 Maintainability & Code Modularity:
- Clean separation of concerns between UI components, FastAPI API routes, NLP utilities, and model loading scripts.

NFR-4 Usability:
- Intuitive, modern design with clear call-to-action buttons, loading spinners during active requests, and unambiguous result banners.

7. Hardware and Software Specifications

7.1 Hardware Requirements

• Processor: Intel Core i3 / AMD Ryzen 3 or higher (Dual-Core 2.0 GHz+).

• System RAM: Minimum 4 GB (8 GB recommended for concurrent backend/frontend execution).

• Disk Storage: Minimum 1 GB of available disk space for dependencies, node_modules, Python virtual environments, and model weights.

7.2 Software Stack

• Operating System: Windows 10/11, macOS, or Linux distribution.

• Client Environment: Modern Web Browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).

• Frontend Stack: ReactJS 18+, Vite Build Tool, Axios / Fetch API, Tailwind CSS / CSS Modules.

• Backend Framework: Python 3.9+, FastAPI, Uvicorn ASGI Server, Pydantic.

• Machine Learning / NLP Ecosystem: Scikit-learn, Pandas, NumPy, NLTK, Joblib / Pickle.

8. Detailed Use Case Analysis

Use Case ID: UC-01

Use Case Name: Analyze News Article Credibility

Primary Actor: End User / Media Student / Fact-Checker

Pre-conditions: The FastAPI backend service and React frontend application are active and connected over the network.

Main Success Scenario:

1. User navigates to the Verity Web Application URL in a browser.
2. User pastes the target news article text into the input field.
3. User clicks the 'Analyze News' button.
4. React frontend validates input and shows a loading indicator.
5. Frontend sends an asynchronous POST payload to `/predict`.
6. FastAPI receives request, applies regex & NLTK preprocessing, and vectorizes text.
7. Model artifact executes inference and outputs prediction ('REAL' / 'FAKE') + probability score.
8. FastAPI sends structured HTTP JSON response.
9. React frontend parses response and renders output card with visual status styling.

Exception Handling:

• Empty Input: UI blocks submission and prompts user to enter text.

• API Server Offline: UI catches network failure and displays an error alert ('Unable to connect to prediction service').

9. System Constraints and Assumptions

Constraints:

• Language Dependency: Current model artifacts and NLTK stop-word lists specifically support English

text inputs.
• Memory Bound: Large transformer-based models require sufficient RAM; local execution uses optimized TF-IDF scikit-learn classifiers.

Assumptions:
• Network Connectivity: Client browser and FastAPI server communicate over HTTP/HTTPS.
• Model Integrity: The serialized `.pkl` file is pre-trained, static, and mounted in the backend service directory.

10. System Interfaces

10.1 User Interface (UI)
• Minimalist SPA dashboard built with React components.
• Real-time input text box, submission trigger, visual loading status, and formatted prediction banner.

10.2 Software / API Interface
• Endpoint: `POST /predict`
• Request Format: `Content-Type: application/json`
Payload: `{"text": "Sample news article text..."}`
• Response Format: `Content-Type: application/json`
Response: `{"prediction": "FAKE", "confidence": 0.91, "status": "success"}`

11. Testing and Verification Strategy

1. Unit Testing: Verification of text preprocessing functions (lowercasing, punctuation stripping) to ensure clean vectorizer inputs.
2. API Endpoint Testing: Validating `/predict` routes using FastAPI Swagger UI (`/docs`) with various JSON payloads.
3. Integration Testing: Verifying end-to-end data transmission between React Axios instances and Uvicorn server.
4. Model Validation: Evaluation on test dataset partitions to verify accuracy, precision, recall, and F1-score metrics.

12. Limitations and Future Improvements

Limitations:
• Scope limited to textual analysis; does not verify embedded images or external hyperlinks.
• Out-of-vocabulary terms outside dataset domain may lower classification confidence.

Future Scope:
• Fine-Tuned Transformer Models: Upgrade from TF-IDF models to BERT / RoBERTa architectures for deeper contextual analysis.

• Automated URL Crawler: Allow users to paste news article links for direct scraping and analysis.
• Multi-Language Support: Incorporate multilingual NLP models for global news verification.
• Database & Analytics Dashboard: Add database persistence to track trends in fake news topics over time.